

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Semestrálna práca z predmetu

Algoritmy a údajové štruktúry 2

SEMESTRÁLNA PRÁCA 1

Vypracoval: Juraj Nižnanský

Študijná skupina: 5ZIB11

Akademický rok: 2026/2027

V Žiline dňa 8.11.2026

Obsah

Programové moduly 2
Spolupráca Presenter-View 3
Spolupráca Presenter-Model 5
Ovládanie aplikácie 6
Intervalové vyhľadávanie 9
KD-Strom 9
Quad-Strom 11
Testovanie a porovnanie údajových štruktúr 13
Výsledok testovania 14
Problémy zistené pri používaní stiahnutých údajových štruktúr 14
Zmenené časti kódov 16
Refactoring testera 16
Zdroje 17

Programové moduly

V kapitole sú popísané vzťahy implementovaných modulov pomocou UML diagramov.

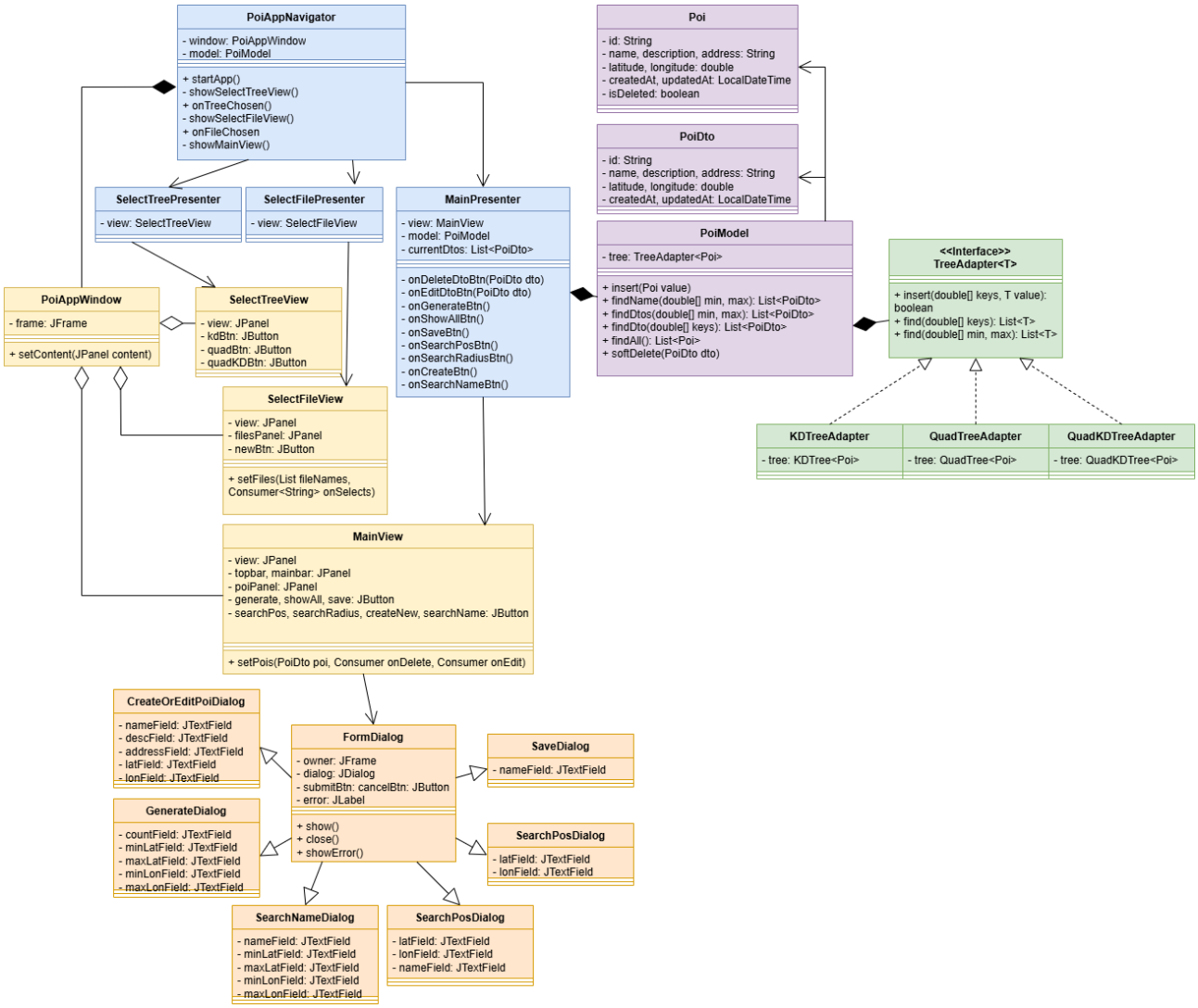
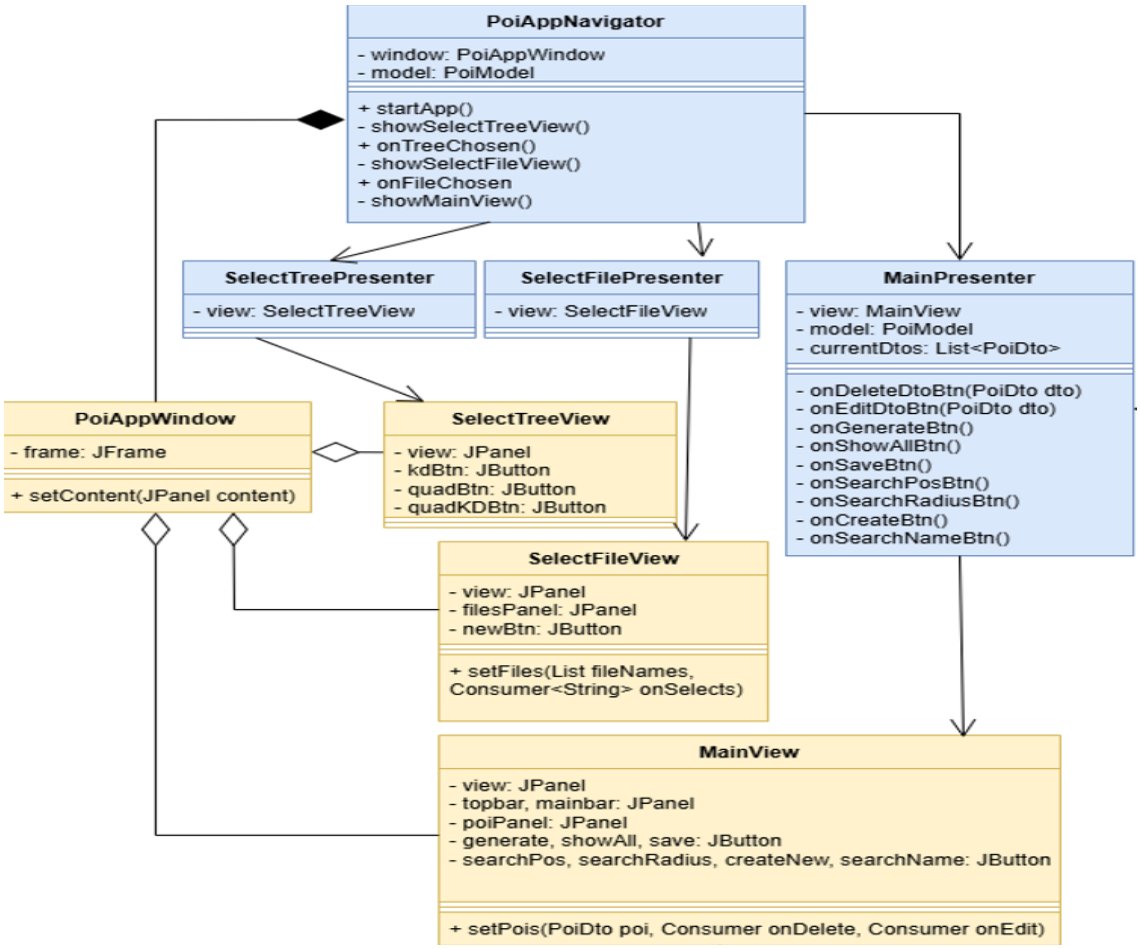


Diagram obsahuje najdôležitejšie triedy spolu s najdôležitejšími atribútmi a metódami a je rozdelený podľa farieb na jednotlivé časti nasledovne:

1. **Žltá farba:** balíček view/views – triedy, ktoré pracujú s knižnicou java.swing a vykresľujú používateľské rozhranie
2. **Oranžová farba:** balíček view/dialogs – tiredy, ktoré zobrazujú formuláre
3. **Modrá farba:** balíček presenter – triedy, ktoré sprostredkúvajú komunikáciu medzi používateľským rozhraním a modelom
4. **Fialová farba:** balíček model – triedy, ktoré slúžia na prácu s dátami a ich ukladanie v pamäti
5. **Zelená farba:** balíček model/adapter – triedy, ktoré používajú návrhový vzor adapter na zjednotenie rozhrania implementovaných stromových štruktúr

Spolupráca Presenter-View



PoiAppNavigator:

Trieda, spravujúca, ktorý view sa má zobraziť (t.j. stará sa o navigáciu v aplikácii). Taktiež vytvára jeden z troch prezenterov, na základe toho, kde sa používateľ v aplikácii nachádza. Každý z prezenterov obsahuje metódy potrebné na vykonávané akcie v korešpondujúcich views.

PoiAppWindow:

Obsahuje hlavné okno aplikácie (JFrame), do ktorého sa vkladajú jednotlivé views.

SelectTreeView:

Reprezentuje prvú obrazovku aplikácie, kde si používateľ vyberá, ktorú údajovú štruktúru použije na ukladanie dát.

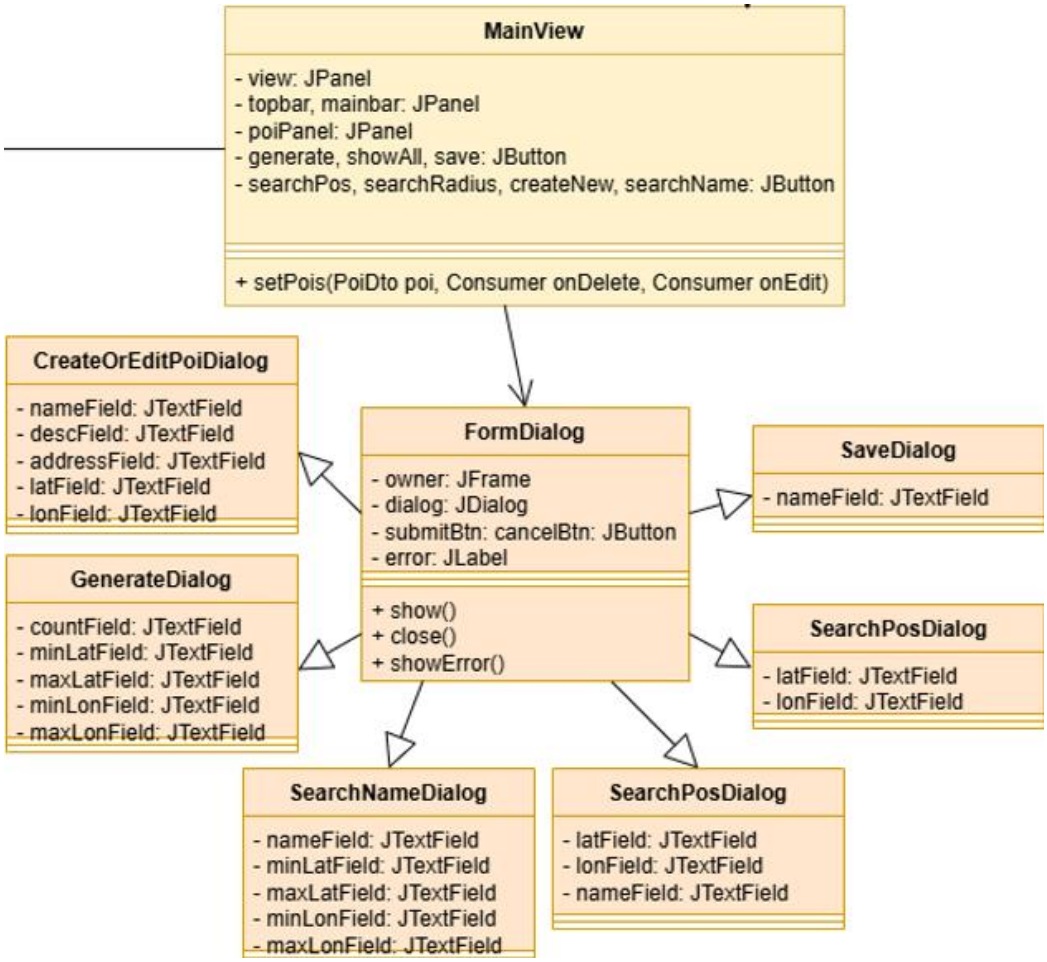
SelectFileView:

Reprezentuje druhú obrazovku aplikácie, kde si používateľ môže zvoliť súbor s uloženými dátami na načítanie, poprípade novú evidenciu.

MainView:

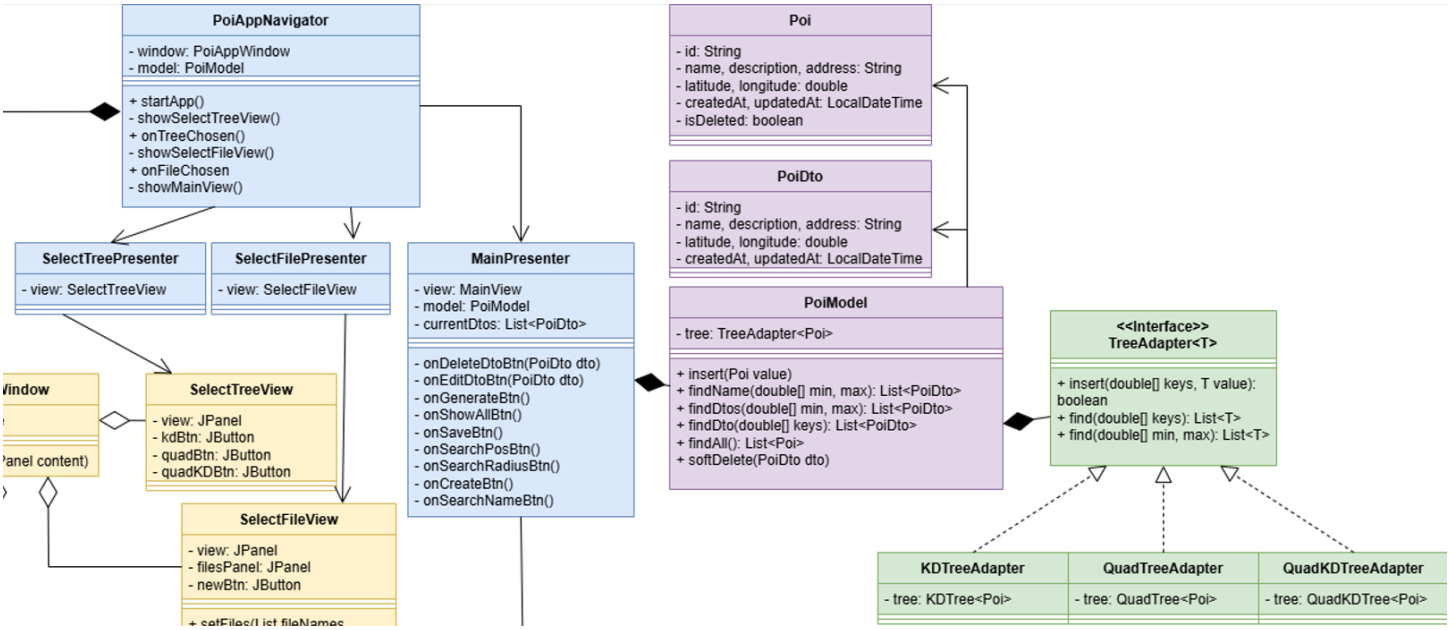
Reprezentuje hlavnú obrazovku aplikácie. Používateľ v nej využíva všetky implementované funkcie podľa zadania.

Formuláre vo forme vyskakovacích dialógov:



Pri vykonávaní určitých akcií na hlavnej obrazovke sa zobrazujú vyskakovacie dialógy. Každý z dialógov má vlastné polia, ktoré používateľ vypĺňa. Na základe vyplnených polí sa daná akcia spracuje. Napr. keď používateľ vykoná akciu uloženia aktuálneho stavu, najprv musí vyplniť formulár, kde zadá názov súboru, do ktoré ho sa majú dáta uložiť.

Spolupráca Presenter-Model



PoiModel:

Je vytvorený navigátorom podľa toho, ktorú údajovú štruktúru si používateľ zvolil na prvej obrazovke. Následne je tento model poslaný do MainPresenter-a, ktorý ho využíva na vykonávanie akcií (t.j. vkladanie dát do údajovej štruktúry a vyhľadávanie dát uložených v štruktúre).

TreeAdapter<T>:

Interface, ktorý plní úlohu návrhového vzoru adapter. Jedná sa o interface, ktorý zjednocuje rozhranie stromov. Tým pádom model nepozná, s ktorou údajovou štruktúrou pracuje, pozná iba rozhranie adaptera.

Poi:

Dátová trieda, ktorá slúži na evidenciu konkrétnych dát (id, názov, popis, adresa, geografická šírka, geografická dĺžka, dátum vytvorenia a dátum úpravy bodu POI).

PoiDto:

Jedná sa o triedu, ktorá je posielaná do používateľského rozhrania na zobrazenie, aby evidované dáta nemohli byť zmenené v GUI.

POI Manager

V kapitole je opísané ako sa výsledná aplikácia zadaná v tretej časti semestrálnej práce ovláda.

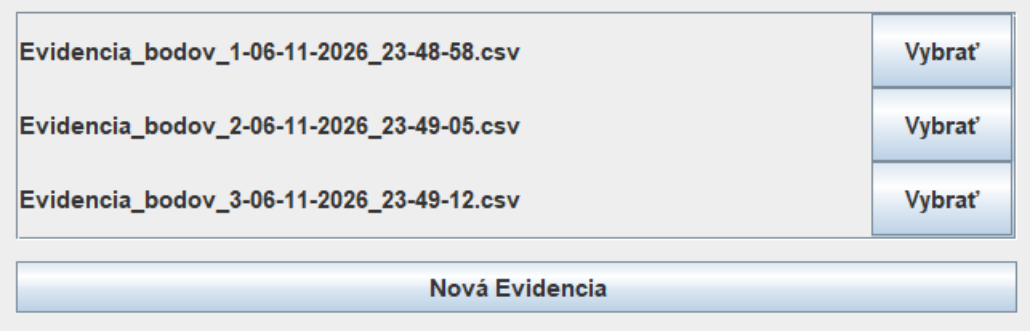
Ovládanie aplikácie

Prvá obrazovka – výber stromu:



Po kliknutí na tlačidlo sa vytvorí strom, s ktorým bude aplikácia pracovať.

Druhá obrazovka – výber súboru / novej evidencie:



Používateľ si môže vybrať, či chce načítať uloženú evidenciu dát, alebo chce založiť novú evidenciu.

Pokiaľ si vyberie súbor, tak sa dáta zo súboru načítajú do vybraného stromu.

Pokiaľ si vyberie novú evidenciu, tak je presunutý na hlavnú obrazovku bez akýchkoľvek dát.

Druhá obrazovka sa zobrazí iba v prípade, že existujú súbory s uloženými dátami.

Hlavná obrazovka – vykonávanie akcií nad bodmi POI:



Používateľ môže vykonávať nasledujúce akcie:

Hľadať Pozíciu:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá geografickú šírku a dĺžku.
- Body POI na tejto pozícii sa zobrazia v tabuľke.

Hľadať Okolie:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá geografickú šírku, dĺžku a polomer v metroch.
- Body POI v okolí zadanej pozície sa zobrazia v tabuľke.

Vytvoriť POI:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá povinný názov, povinný popis, nepovinnú adresu a povinnú geografickú šírku a dĺžku.
- Ak sú zadané informácie platné, vytvorí sa nový bod POI, ktorý sa uloží do evidencie.

Hľadať Názov:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá obdĺžnikovú oblasť a názov, ktorý majú body POI v tejto oblasti obsahovať.
- Ak sa nájdu body, ktoré obsahujú názov, zobrazia sa v tabuľke.

Všetky:

- Po stlačení tlačidla sa vypíšu všetky evidované body POI.

Generovať:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá počet prvkov a obdĺžnikovú oblasť.
- Následne sa vygeneruje zadaný počet prvkov s náhodnými informáciami, ktoré budú mať súradnice v rámci zadanej oblasti.

Uložiť:

- Po stlačení tlačidla sa otvorí formulár, kde používateľ zadá názov súboru.
- Všetky evidované body POI sa zapíšu do súboru v predpísanom formáte.

Upraviť (ikona ceruzky):

- Po stlačení tlačidla sa otvorí formulár, kde používateľ môže zmeniť informácie pre zvolený bod POI.
- Upravený bod POI sa z evidencie vymaže a pridá sa na novo.

Vymazať (ikona X):

- Po stlačení tlačidla sa zvolený bod označí ako vymazaný a ďalej sa v aplikácii nezobrazuje.
- Vymazané body sa do súboru neukladajú ani sa v aplikácii nevyhľadávajú.

Intervalové vyhľadávanie

V kapitole je opísaný princíp fungovania intervalového vyhľadávania v semestrálnej práci implementovaných údajových štruktúrach.

KD-Strom

Implementácia intervalového vyhľadávania je rozdelená na 6 častí nasledovne:

Výmena kľúčov:

Pokiaľ sú minimálne kľúče väčšie ako maximálne kľúče pre jednotlivé úrovne, tak sa vymenia.

```
public LinkedList<T> find(Comparable[] minKeys, Comparable[] maxKeys) {
    // Exchange keys if min > max
    for (int i = 0; i < this._k; i++) {
        if (minKeys[i].compare(maxKeys[i]) > 0) {
            Comparable temp = minKeys[i];
            minKeys[i] = maxKeys[i];
            maxKeys[i] = temp;
        }
    }
```

Strom neobsahuje žiadne hodnoty:

V prípade, že strom neobsahuje žiadne hodnoty (t.j. koreň stromu = NULL), vráti sa prázdny zoznam a algoritmus sa ukončí.

```
LinkedList<T> found = new LinkedList<>();
if (this._root == null) {
    return found;
}
```

Inicializácia zásobníkov a vyhľadávacieho cyklu:

Aby sa vyhlo používaniu rekurzie, uzly stromu, ktoré sa majú prehľadať sa ukladajú do zásobníka. Taktiež sa do zásobníka ukladá a hĺbka, na ktorej sa korešpondujúci uzol nachádza, kvôli znalosti kľúča, na základe ktorého sa vykonáva porovnanie. Vyhľadávanie sa vykonáva pokiaľ nie je zásobník prázdny.

```
LinkedList<KDNode<T>> nodeStack = new LinkedList<>();
LinkedList<Integer> depthStack = new LinkedList<>();

nodeStack.push(this._root);
depthStack.push( e: 0);

while (!nodeStack.isEmpty()) {
    KDNode<T> current = nodeStack.pop();
    int depth = depthStack.pop();
    int keyIndex = depth % this._k;

    // Accepting whole subtree: lies completely inside
    Comparable currentKey = current.getKey(keyIndex);
```

Zamietnutie pravého podstromu:

Pokiaľ je maximálny vyhľadávaný kľúč na danej úrovni menší, ako kľúč prehľadávaného uzla, nie je potrebné prehľadávať pravý podstrom (v ktorom sú vždy väčšie kľúče) – algoritmus sa potrebuje dostať k menším kľúčom.

```
// Left Subtree MAX < Key - Reject right subtree
if (maxKeys[keyIndex].compare(currentKey) == -1) {
    KDNode<T> leftSon = current.getLeftSon();
    if (leftSon != null) {
        nodeStack.push(leftSon);
        depthStack.push( e: depth + 1);
    }
}
```

Zamietnutie ľavého podstromu:

Pokiaľ je minimálny vyhľadávaný kľúč na danej úrovni väčší, ako kľúč prehľadávaného uzla, nie je potrebné prehľadávať ľavý podstrom (v ktorom sú vždy menšie alebo rovné kľúče) – algoritmus sa potrebuje dostať k väčším kľúčom.

```
// Right Subtree MIN > Key - Reject left subtree
else if (minKeys[keyIndex].compare(currentKey) == 1) {
    KDNode<T> rightSon = current.getRightSon();
    if (rightSon != null) {
        nodeStack.push(rightSon);
        depthStack.push( e: depth + 1);
    }
}
```

Prehľadávanie oboch podstromov:

Pokiaľ sa kľúč aktuálneho uzla na danej úrovni nachádza vo vyhľadávanom intervale, skontroluje sa, či sa má aktuálny uzol pridať do zoznamu vyhľadaných (na základe kľúčov všetkých úrovní) a prehliadka pokračuje v ľavom aj pravom podstrome.

```
// Both Subtrees Key: <MIN, MAX>
else {
    if (this.nodeInInterval(current, minKeys, maxKeys)) {
        found.add(current.getData());
    }

    KDNode<T> rightSon = current.getRightSon();
    if (rightSon != null) {
        nodeStack.push(rightSon);
        depthStack.push( e: depth + 1);
    }
    KDNode<T> leftSon = current.getLeftSon();
    if (leftSon != null) {
        nodeStack.push(leftSon);
        depthStack.push( e: depth + 1);
    }
}
```

Quad-Strom

Implementácia intervalového vyhľadávania je rozdelená na 4 časti nasledovne:

Výmena kľúčov:

Pokiaľ sú minimálne kľúče väčšie ako maximálne, tak sa vymenia.

```
public LinkedList<T> find(QuadKey xMinKey, QuadKey xMaxKey, QuadKey yMinKey, QuadKey yMaxKey) {
    // Exchange if min > max
    if (xMinKey.compare(xMaxKey) > 0) {
        QuadKey temp = xMaxKey;
        xMaxKey = xMinKey;
        xMinKey = temp;
    }
    if (yMinKey.compare(yMaxKey) > 0) {
        QuadKey temp = yMaxKey;
        yMaxKey = yMinKey;
        yMinKey = temp;
    }
```

Inicializácia zásobníka a vyhľadávacieho cyklu:

Aby sa vyhlo používaniu rekurzie, uzly stromu, ktoré sa majú prehľadať sa ukladajú do zásobníka. Vyhľadávanie sa vykonáva pokiaľ nie je zásobník prázdny.

```
LinkedList<T> found = new LinkedList<>();

LinkedList<QuadNode<T>> nodeStack = new LinkedList<>();
nodeStack.push(this._root);
while (!nodeStack.isEmpty()) {
    QuadNode<T> current = nodeStack.pop();
```

Kontrola, či aktuálny uzol obsahuje dáta:

Pokiaľ aktuálny uzol obsahuje dáta, skontroluje sa pre každý záznam, či sa nachádza vo vyhľadávanom intervale. V prípade, že sa v ňom nachádza, je pridaný do zoznamu vyhľadaných. Ak je aktuálny uzol listom stromu, po prehľadaní jeho dát cyklus pokračuje na ďalší uzol.

```
if (current.hasData()) {
    // Check for data to be added to found
    for (var data : current.getData()) {
        if (data.xKey().compare(xMinKey) >= 0
                && data.xKey().compare(xMaxKey) <= 0
                && data.yKey().compare(yMinKey) >= 0
                && data.yKey().compare(yMaxKey) <= 0
        ) {
            found.add(data.getData());
        }
    }
}

if (current.isLeaf())
    continue;
```

Prehľadávanie podstromov:

Podľa hraničných bodov prehľadávanej oblasti sa zistí, ktoré podstromy aktuálneho uzla sa majú prehľadať. Dané vrcholy podstromov sa pridajú do zásobníka. Implementácia zabezpečuje, že žiadny podstrom nie je prehľadaný viac ako jedenkrát.

```
int[] indices = {
        current.getQuadrantIndex(xMinKey, yMinKey),
        current.getQuadrantIndex(xMaxKey, yMinKey),
        current.getQuadrantIndex(xMinKey, yMaxKey),
        current.getQuadrantIndex(xMaxKey, yMaxKey)
};
boolean[] sonsPushed = new boolean[4];
for (int index : indices) {
    if (!sonsPushed[index]) {
        sonsPushed[index] = true;
        nodeStack.push(current.getSon(index));
    }
}
```

Testovanie a porovnanie údajových štruktúr

V kapitole je opísané, akým spôsobom bolo vykonávané testovanie na troch údajových štruktúrach, ktoré boli implementované v semestrálnej práci, a šiestich stiahnutých údajových štruktúrach.

Použité stiahnuté údajové štruktúry boli:

- **KD-Strom Oschrenk [1]:** na GitHube 17 hviezdičiek a 7 forkov
- **KD-Strom z knižnice Tinspin-indexes [2]:** na GitHube 117 hviezdičiek a 7 forkov
- **KD-Strom Geo-Assist [3]:** na GitHube 216 hviezdičiek a 21 forkov
- **Quad-Strom Varunpant [4]:** na GitHube 116 hviezdičiek a 45 forkov
- **Quad-Strom Jotschi [5]:** na GitHube 22 hviezdičiek a 7 forkov
- **Quad-Strom z knižnice Tinspin-indexs [2]:** na GitHube 117 hviezdičiek a 7 forkov

Pri testovaní údajových štruktúr boli vykonané nasledovné testy:

1. **Test vkladania a vyhľadávania:** v teste sa vykonalo celkovo 100000 operácií náhodne vybraných (so šancou 50%) medzi operáciou vloženia a operáciou vyhľadávania. Operácia vloženie do stromu vložila náhodné hodnoty kľúčov. Operácia hľadania vyhľadávala náhodne uložené oblasti a pre každú oblasť skontrolovala, či sa našli správne výsledky. Test bol vykonaný aj na duplicitných kľúčoch.
2. **Test, či štruktúra obsahovala všetky vložené kľúče:** v teste sa vykonalo 100000 operácií vloženia náhodných kľúčov. Následne boli vyhľadané všetky hodnoty uložené v štruktúre. Test skontroloval, či boli nájdené všetky výsledky. Test bol vykonaný aj na duplicitných kľúčoch.
3. **Test vymazania:** v teste sa vykonalo 100000 operácií vloženia náhodných kľúčov. Následne boli vymazávané náhodné hodnoty z referenčného zoznamu. Test skontroloval, či neboli vymazané iné hodnoty ako určené. Test skontroloval, či boli vymazané hodnoty naozaj vymazané. Test bol vykonaný aj na duplicitných kľúčoch.
4. **Výkonnostné testovanie operácie vloženia:** v teste boli do štruktúry vkladané všetky dáta načítané zo súboru ‚slovakia_nodes.csv'. Na zistenie rýchlosti operácie bol využitý profiler v prostredí Apache NetBeans.
5. **Výkonnostné testovanie operácie vyhľadávania:** test vykonal intervalové vyhľadávanie na štruktúrach podľa načítaných bodov zo súboru ‚testovacie_oblasti.csv'. Na zistenie rýchlosti operácie bol využitý profiler v prostredí Apache NetBeans.
6. **Doplnkové testovanie Quad-Stromu od používateľa Jotschi:** doplnkové testovanie bolo vykonané z dôvodu, že tento strom neumožňoval operáciu intervalového vyhľadávania, a preto nebolo možné štruktúru otestovať ako ostatné.

Výsledok testovania

Výsledok testovania je zaznamenaný v tabuľke nižšie.

Štruktúry implementované v semestrálnej práci:

Všetky 3 štruktúry splnili navrhnuté testy, a to v prípade scenára s duplicitnými kľúčmi aj bez duplicitných kľúčov. Operácia vymazania nie je na štruktúrach implementovaná, a preto ani testy na túto operáciu neboli vykonané.

Stiahnuté implementácie štruktúr:

Výsledky testov a problémy so stiahnutými implementáciami údajových štruktúr sú popísané v kapitole nižšie.

		Testovanie bez duplicitných kľúčov			Testovanie s duplicitnými kľúčmi			Výkonnostné testovanie	
		1. Test	2. Test	3. Test	1. Test	2. Test	3. Test	4. Test	5. Test
		Vkladanie / Vyhľadávan	Obsahuje vložené	Vymazanie	Vkladanie / Vyhľadávan	Obsahuje vložené	Vymazanie	Vloženie	Vyhľadávanie
Implementované v semestrálnej práci	KD-Strom	**Prešiel**	**Prešiel**	Neumožňuje	**Prešiel**	**Prešiel**	Neumožňuje	**1698 ms**	**6532 ms**
	Quad-Strom	**Prešiel**	**Prešiel**	Neumožňuje	**Prešiel**	**Prešiel**	Neumožňuje	**236 ms**	**744 ms**
	Quad-KD-Strom	**Prešiel**	**Prešiel**	Neumožňuje	**Prešiel**	**Prešiel**	Neumožňuje	**530 ms**	**1938 ms**
GitHub implementácie KD-Stormu	KD-Strom používateľa Oschrenk [1]	**Prešiel**	**Prešiel**	**Prešiel**	**Neprešiel**	**Neprešiel**	**Neprešiel**	**4083 ms**	**4027 ms**
	KD-Strom používateľa Tzaeschke [2]	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**1039 ms**	**7036 ms**
	KD-Strom Geo-Assist [3]	**Prešiel**	**Prešiel**	Umožňuje iba podľa ID	**Neprešiel**	**Neprešiel**	**Neprešiel**	**1348 ms**	**3779 ms**
GitHub implementácie Quad-Stromu	Quad-Strom používateľa Varunpant [4]	**Prešiel**	**Prešiel**	**Prešiel**	**Neprešiel**	**Neprešiel**	**Neprešiel**	**627 ms**	**2350 ms**
	Quad-Strom používateľa Jotschi [5]	**Doplnkové testovanie z dôvodu absencie intervalového**			**Doplnkové testovanie z dôvodu absencie intervalového**			**130 ms**	**Neumožňuje**
	Quad-Strom používateľa Tzaeschke [2]	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**Prešiel**	**637 ms**	**2191 ms**

Problémy zistené pri používaní stiahnutých údajových štruktúr

KD-Strom používateľa Oschrenk:

- Neumožňuje vkladanie duplicitných kľúčov
    - Odmieta rovnaký kľúč výnimkou
- Nie je zaistená univerzálnosť kľúča – je možné vkladať iba double hodnoty

KD-Strom Geo-Assist:

- Nie je zaistená univerzálnosť kľúča
- Jedná sa o 2D-Strom, nemá K dimenzií
- Umožňuje vymazanie iba podľa ID, ktoré sa nastavuje pri vkladaní bodu
- Umožňuje vkladať iba hodnoty od x = <-90, 90> a y = <-180, 180>, nakoľko sa pravdepodobne jedná o asistenčnú knižnicu pri práci s geografickými údajmi
- Neumožňuje vkladanie duplicít

Quad-Strom používateľa Varunpant:

- Nie je zaistená univerzálnosť kľúča
- Využíva sa rekurzia
- Neumožňuje vkladanie duplicitných kľúčov – pri vložení rovnakého kľúča prepíše pôvodnú hodnotu na novú

Quad-Strom používateľa Jotschi:

- Využitá bola verzia, kde sa vkladajú body so súradnicou x a y
- Umožňuje vkladať iba súradnice typu long → všetky generované hodnoty double, ako aj lon a lat zo zadaných súborov boli prenásobené hodnotou 1‘000‘000‘000
- Využíva rekurziu
- Implementácia neumožňuje intervalové vyhľadávanie
- Implementácia neumožňuje operáciu mazania
- Vyhľadávanie metódou getElements vrátilo všetky záznamy, ktoré sa nachádzali v liste – je ich preto potrebné opäť odfiltrovať
- **Doplnkové testovanie kvôli absencii intervalového vyhľadávania:**
    - Vyhľadanie všetkých uzlov zo zadaného súboru: test prešiel
    - Vloženie duplicitných hodnôt a ich následné vyhľadanie: test prešiel čiastočne (pri vyhľadaní bolo nájdených viac hodnôt, ktoré sa nachádzali v danom liste, bolo ich potrebné odfiltrovať)

Knižnica Tinspin-indexes od používateľa Tzaeschke:

- Knižnica fungovala a splnila všetky testy
- Pri intervalovom vyhľadávaní sa vracia iterátor, cez ktorý sa následne robí prehliadka

Zmenené časti kódov

V kapitole sú opísané zmeny v kódoch od priebežných kontrol semestrálnej práce.

Refactoring testera

Pôvodný tester obsahoval samostatné testovacie triedy pre každú štruktúru, ktorú bolo potrebné otestovať. V daných triedach boli v zásade totožné kódy, no vždy s inou štruktúrou, ktorá bola testovaná. Pre zamedzenie duplicity kódu a dodržanie DRY princípu (angl. „do not repead yourself“ – „neopakuj sa“) bol aplikovaný návrhový vzor **Adapter.**

Tento prístup zvýšil prehľadnosť, čitateľnosť a udržateľnosť testera.

Taktiež bola pridaná metóda na testovanie operácie **mazania**, kvôli testovaniu údajových štruktúr zo zadania 2.

Zdroje

[1] Implementácia KD-Stromu z GitHub repozitára od používateľa Oschrenk.

Link: https://github.com/oschrenk/kd-tree

[2] Knižnica Tinspin-indexes z GitHub repozirára od používateľa Tzaeschke

Link: https://github.com/tzaeschke/tinspin-indexes/tree/master

[3] Implementácia KD-Stromu z GitHub repozitára geo-assist

Link: https://github.com/thegeekyasian/geo-assist/tree/master

[4] Implementácia Quad-Stromu z GitHub repozitára od používateľa Varunpant

Link: https://github.com/varunpant/Quadtree/tree/master

[5] Implementácia Quad-Stromu z GitHub repozitára od používateľa Jotschi

Link: https://github.com/metaloom/JavaQuadTree/tree/master